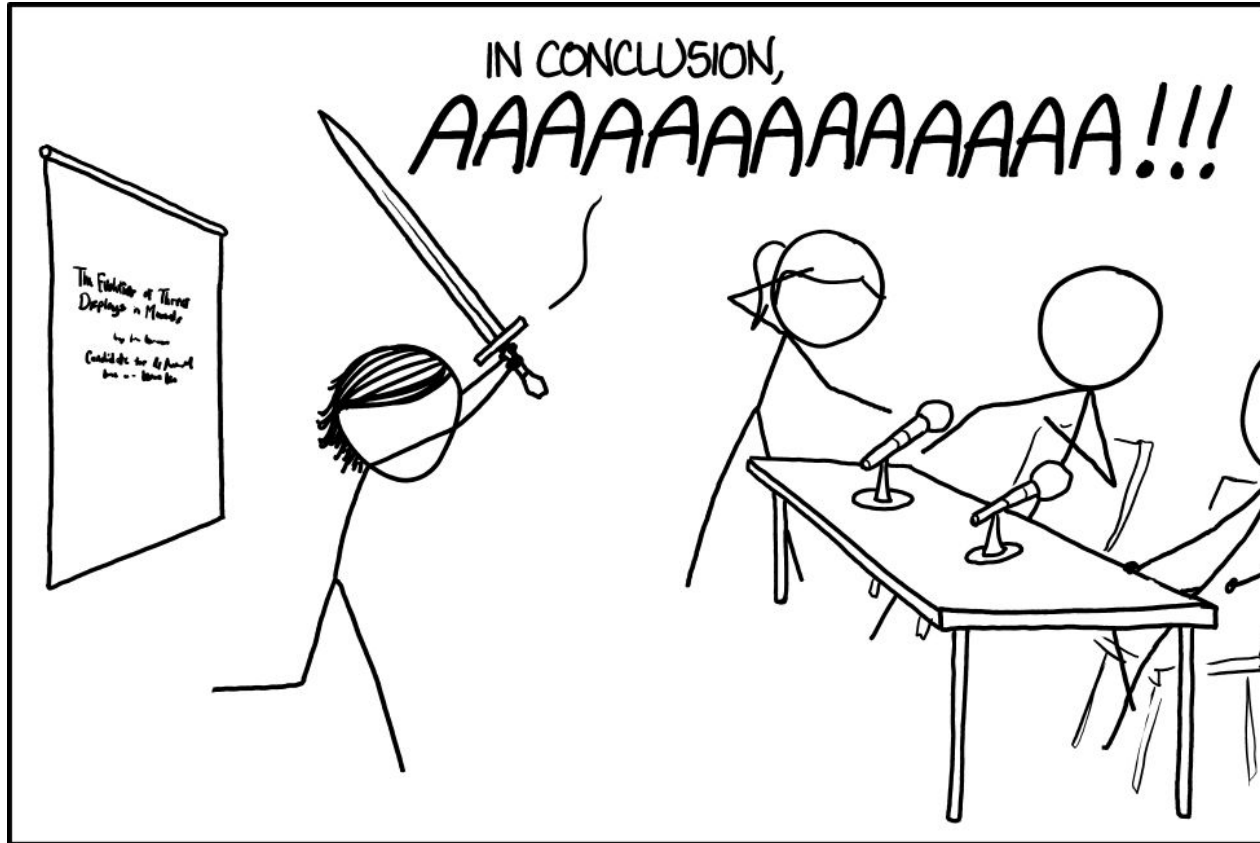
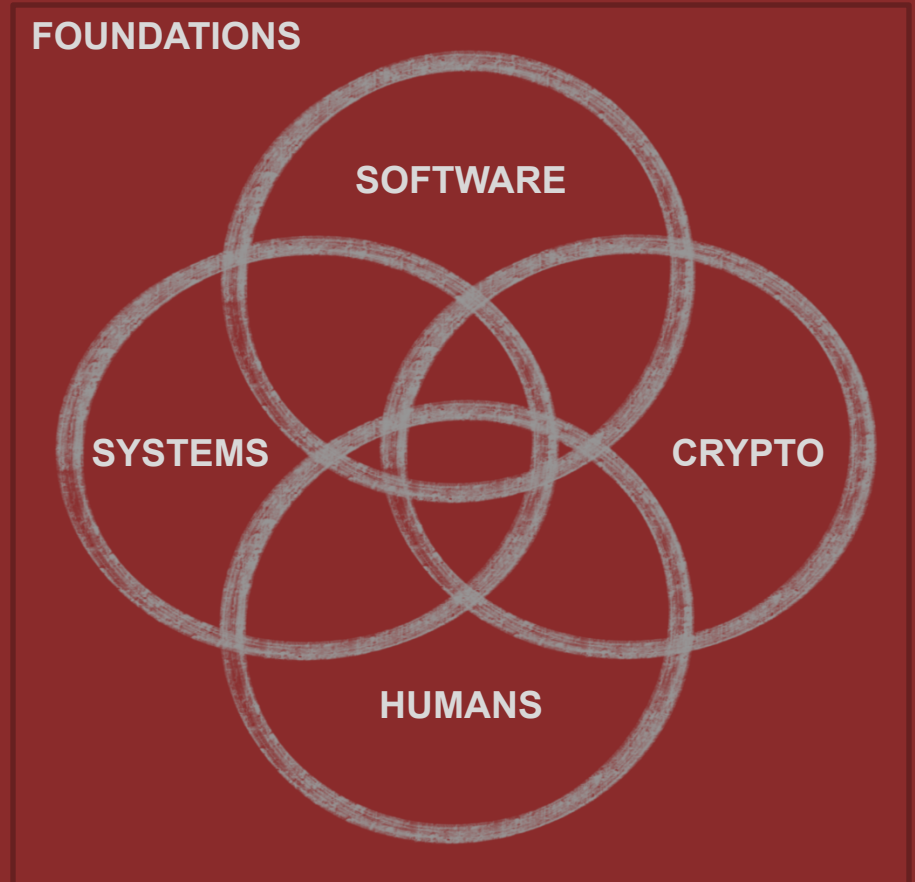


Διάλεξη #7 - Mitigations



THE BEST THESIS DEFENSE IS A GOOD THESIS OFFENSE.

Huge thank you to [David Brumley](#) from Carnegie Mellon University for the guidance and content input while developing this class

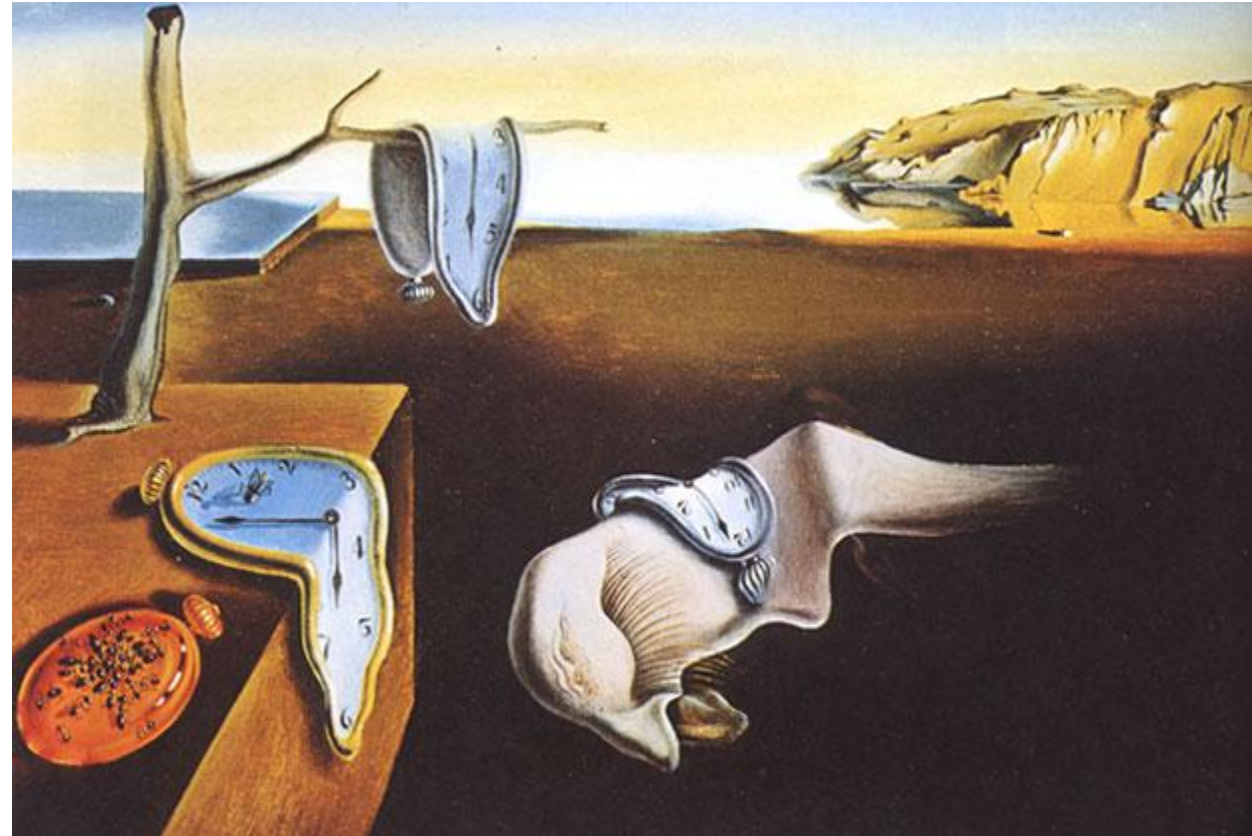


Ανακοινώσεις / Διευκρινίσεις

- Υπενθύμιση: Η εργασία #0 και το bonus #0 κλείνουν σήμερα.
- Guest Lectures Tomorrow!
- OpenDays Tomorrow!

Την Προηγούμενη Φορά

1. Format String Attacks and review
2. Application Security



Σήμερα

- Application Security Continued
 - Supply Chain (SBOM)
- Control Flow Hijack Mitigations





Security in the News

CISA Flags SolarWinds, Ivanti, and Workspace One Vulnerabilities as Actively Exploited

👤 Ravie Lakshmanan 📅 Mar 10, 2026



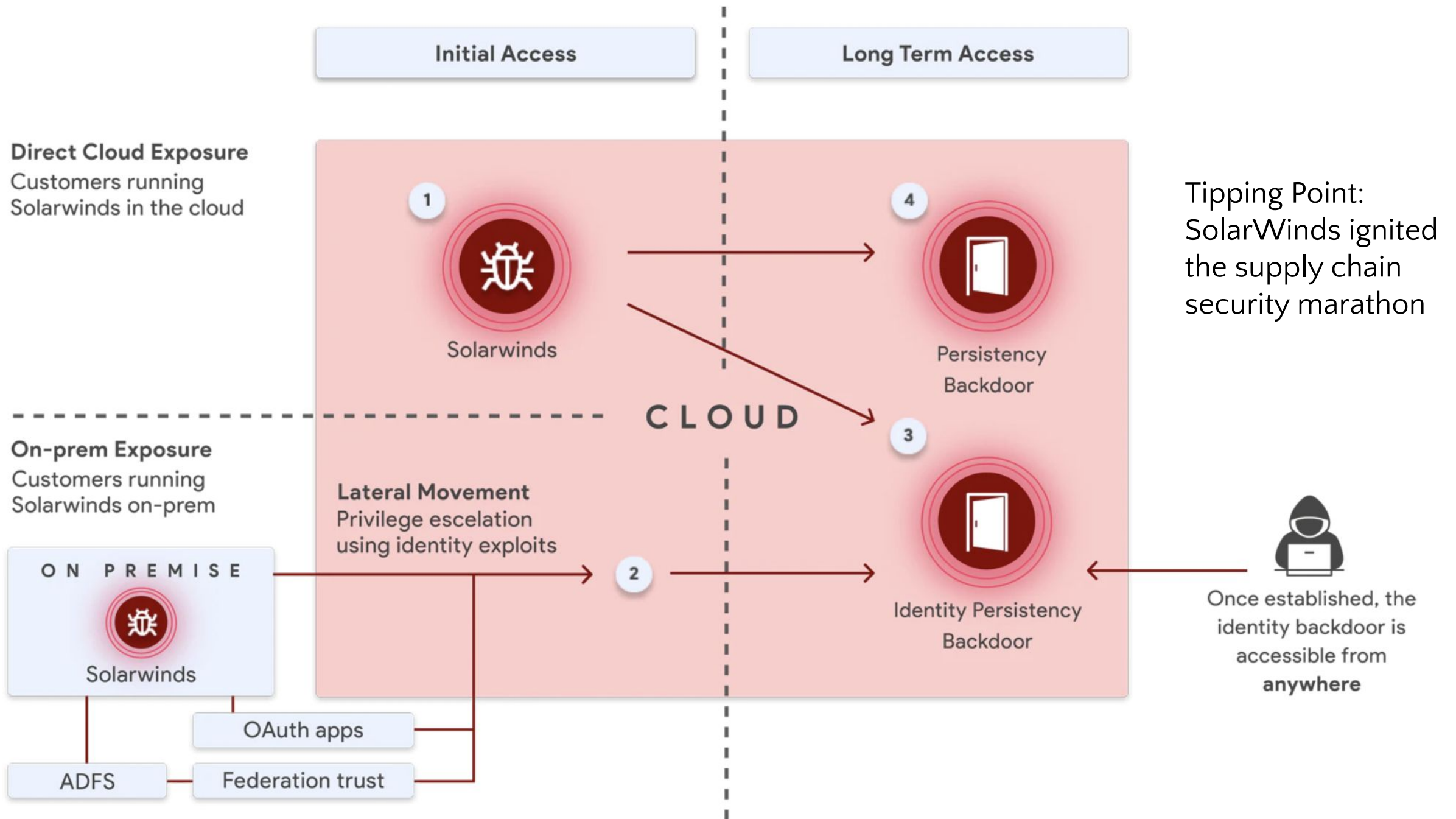
Known Exploited Vulnerabilities Catalog

KEV has ~1,500 CVEs total

The U.S. Cybersecurity and Infrastructure Security Agency (CISA) on Monday added three security flaws to its Known Exploited Vulnerabilities (KEV) catalog, based on evidence of active exploitation.

The vulnerability list is as follows -

- [CVE-2021-22054](#) (CVSS score: 7.5) - A server-side request forgery ([SSRF](#)) vulnerability in Omnisia Workspace One UEM (formerly VMware Workspace One UEM) that could allow a malicious actor with network access to UEM to [send requests without authentication](#) and to gain access to sensitive information.





Application Security Today

The Spectrum of Security Risks

“Unknown unknowns”

Risks that cannot be identified.

The quantity of unknown unknown threats is unknown.

“Known unknowns”

Identifiable risks that could potentially lead to a compromise. (CWE)

There are anywhere between 10,000s to 1Ms.

“Known knowns”

Identifiable risks that are known to lead to compromise. (CVE)

There are anywhere are 100s to 1,000s.

Is Your System Vulnerable?

Where would you start to answer this question?

For years, we had no place to start...

Static Vulnerability Scanner Strategy

Step 1: Build asset inventory (packages, libraries, etc) and find out their versions ([CPE, PURL, etc](#))

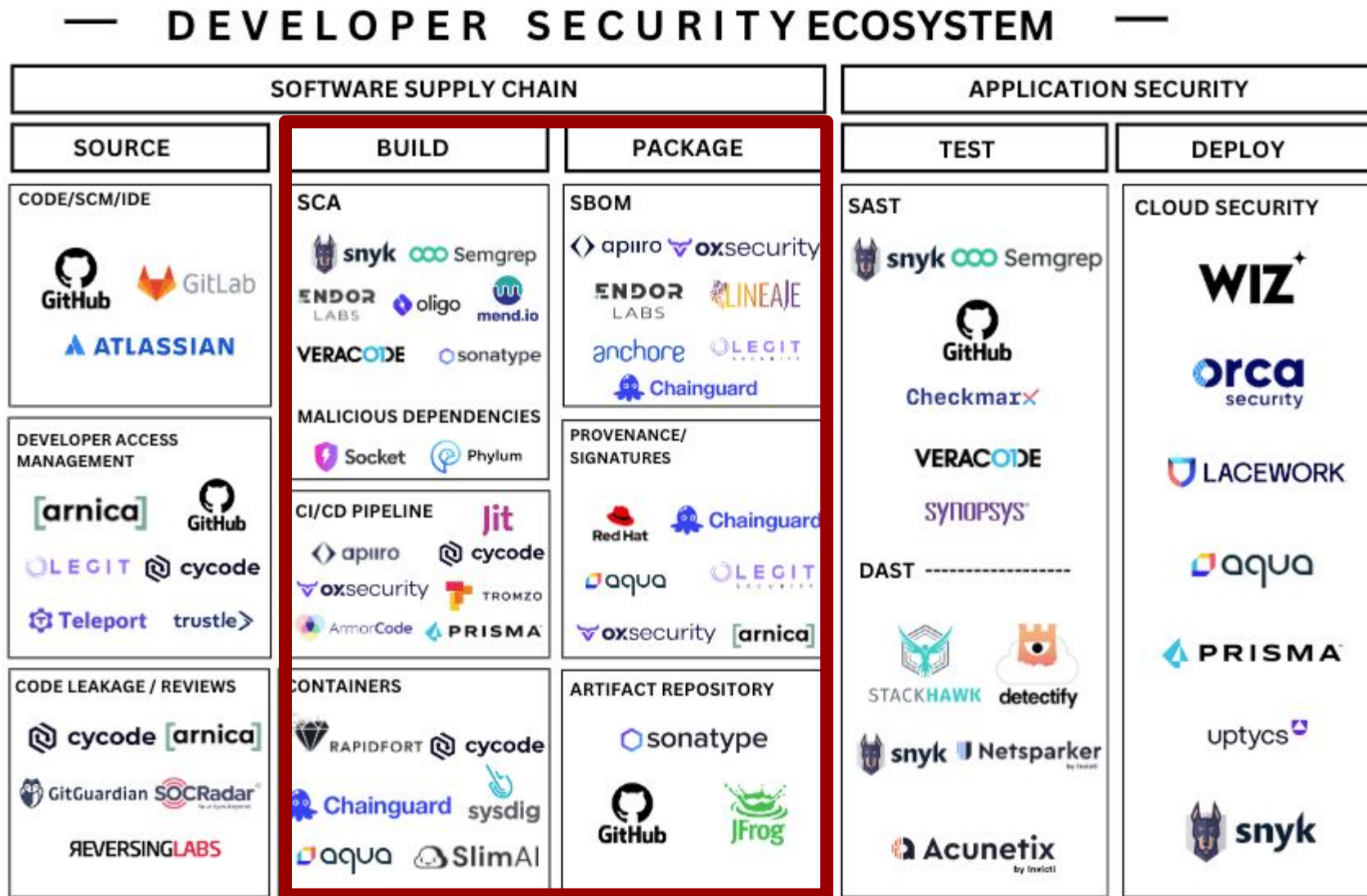
Step 2: Check whether they are known vulnerable (CVE)

Step 3: Remediate vulnerable packages identified in step 2 and update inventory

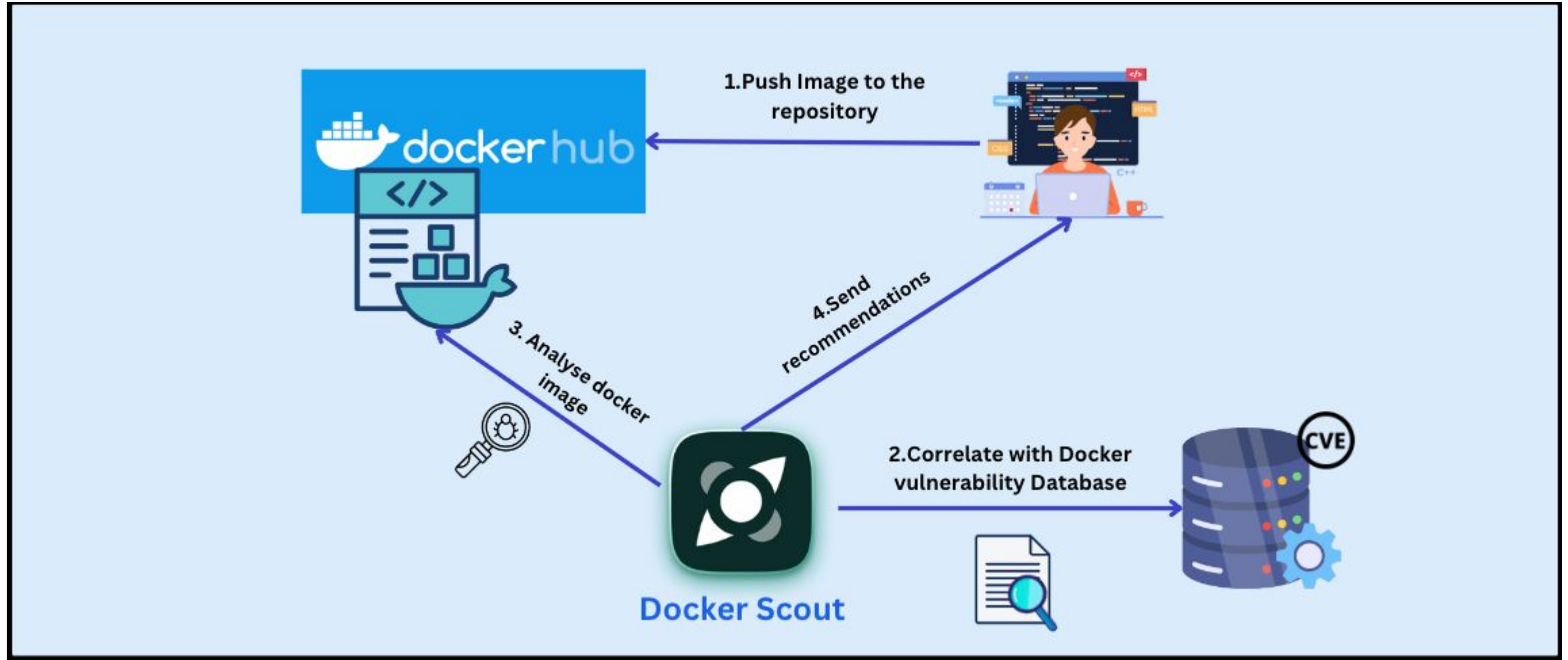
How would you apply the strategy above in your organization?

Automate with Tools

Known as Software Composition Analysis (SCA) or Software Bill of Materials (SBOM)



One Example: Docker Scout



Note: you can analyze anyone's code - is this an advantage?

Static Vulnerability Scanner Strategy

Step 1: Build asset inventory (packages, libraries, etc) and find out their versions ([CPE, PURL, etc](#))

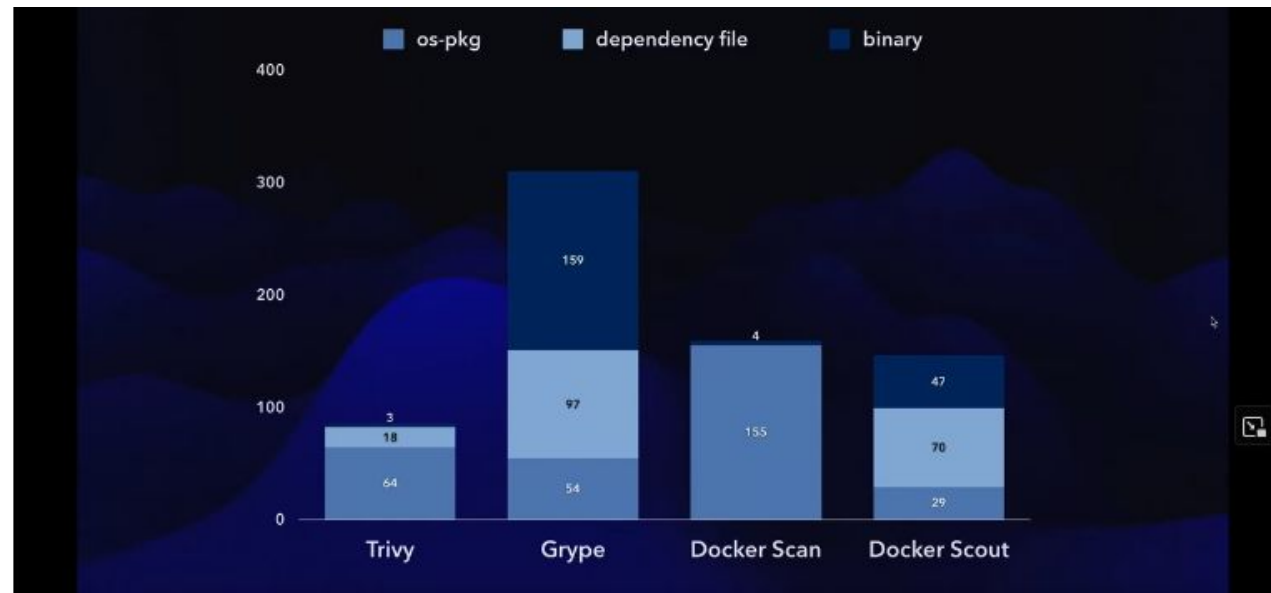
Step 2: Check whether they are known vulnerable (CVE)

Step 3: Remediate vulnerable packages identified in step 2 and update inventory

I scanned my system with docker scout and get a clean report - am I secure?

Security Challenges

- TOCTOU – CVEs are evolving rapidly. The second after release a new vulnerability affecting you may be out there
- Scanners are incomplete (first party code) and easy to [trick](#)
- Some findings may be "false positives" (more on this later)





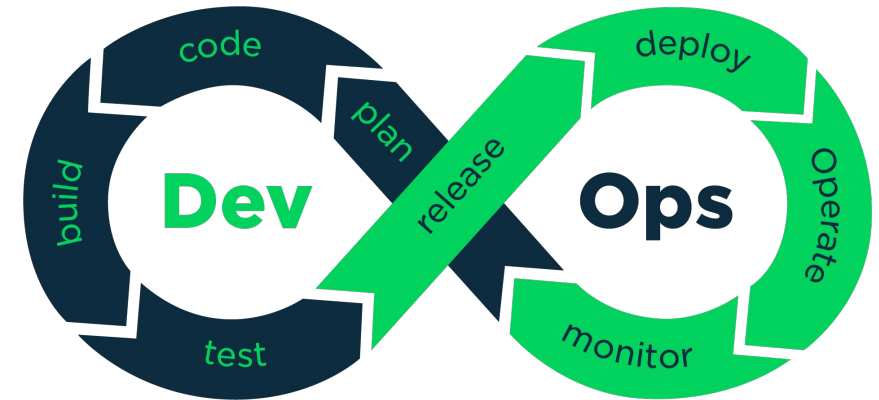
Dev(Sec)Ops

Interview Question: What is DevOps?

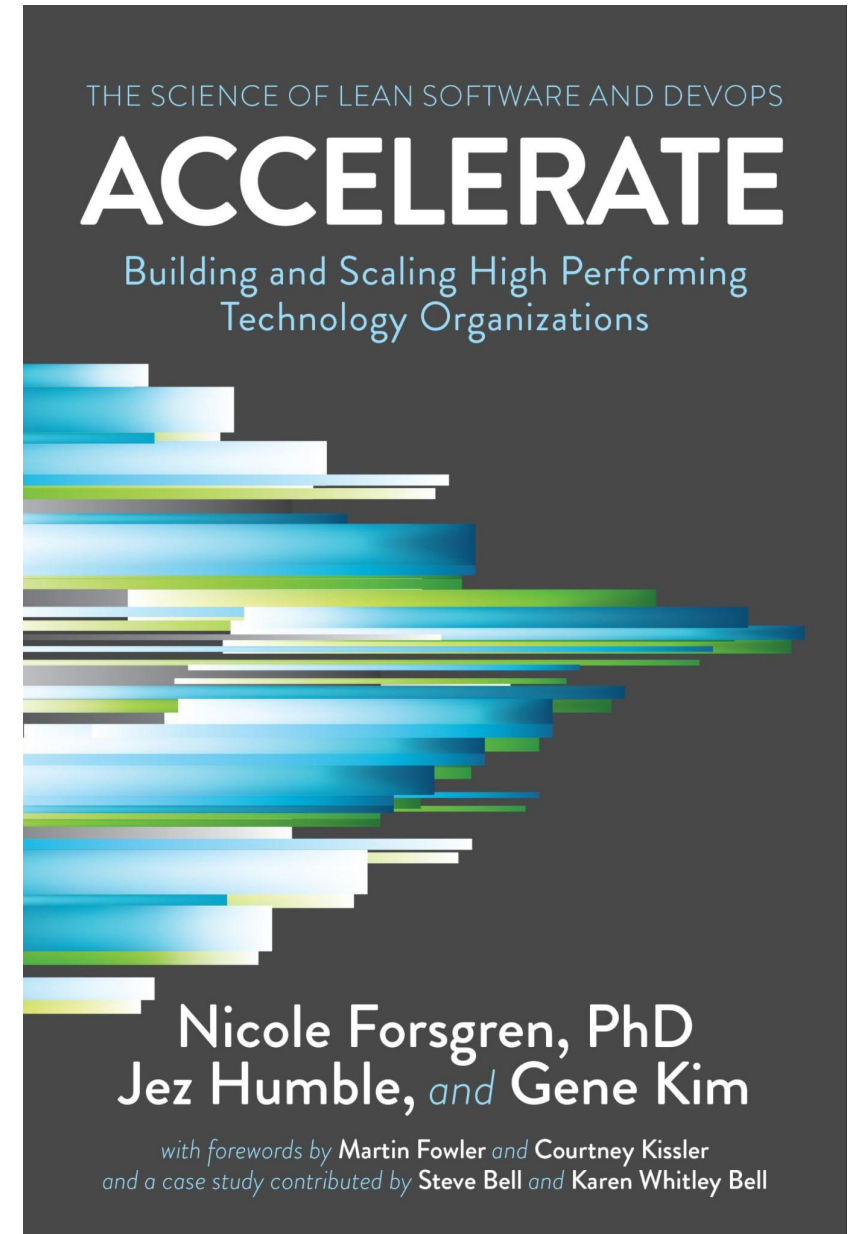


DevOps: Development + Operations

- DevOps is the integration and automation of the software development and information technology operations
- Highly-respected role in the industry
 - Site-reliability engineering
 - Developer productivity
- Typically has a multiplicative effect



The DevOps Textbook that started it all (2018)



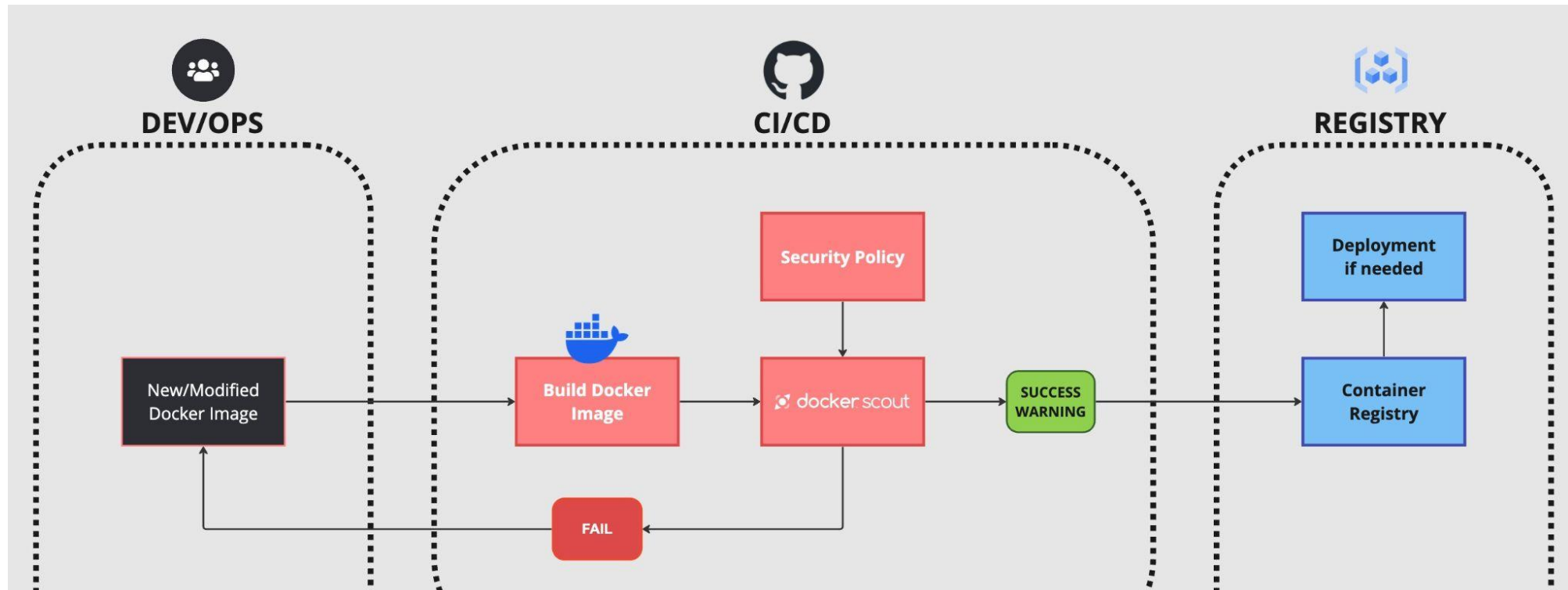
Key Insight: 4 Metrics Matter (DORA)

Software delivery performance metric	Elite	High	Medium	Low
 Deployment frequency For the primary application or service you work on, how often does your organization deploy code to production or release it to end users?	On-demand (multiple deploys per day)	Between once per week and once per month	Between once per month and once every 6 months	Fewer than once per six months
 Lead time for changes For the primary application or service you work on, what is your lead time for changes (i.e., how long does it take to go from code committed to code successfully running in production)?	Less than one hour	Between one day and one week	Between one month and six months	More than six months
Time to restore service For the primary application or service you work on, how long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?	Less than one hour	Less than one day	Between one day and one week	More than six months
Change failure rate For the primary application or service you work on, what percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	0%-15%	16%-30%	16%-30%	16%-30%

Common Practice Today: Continuous Integration (CI)

Continuous integration (CI) ensures that the entire codebase is in a good state.

When is the best time to check your software?



Example Repo



Defenses and Mitigations

Defenses

1. Canaries
2. DEP (Data Execution Prevention) / NX (No Execute)
3. ASLR (Address Space Layout Randomization)

Defenses we will see today focus on
preventing control hijacks (Prevention)

Why does this program dump core even though we don't overwrite the return address?

```
int is_good() {  
    char * magic = "8675309";  
    char buf[32];  
    fread(buf, 128, 1, stdin); // BOFs are cool  
    if (strncmp(magic, buf, strlen(magic)) == 0) {  
        return 1;  
    }  
    return 0;  
}
```

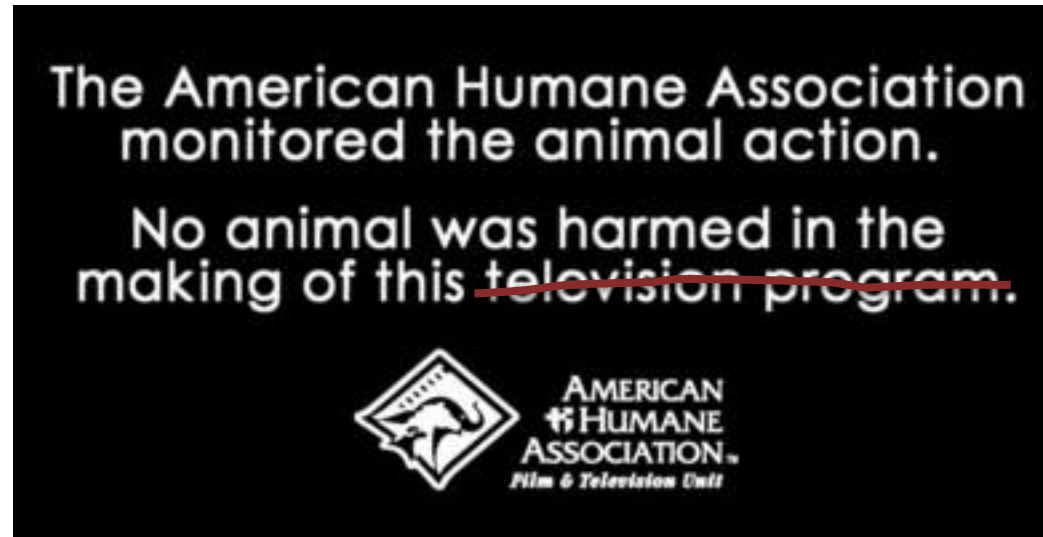
Can we control the instruction pointer in such a program? Yes / No? Why?

Introducing Canary / Stack Cookies



What Is a “Canary”?

Wikipedia: “the historic practice of using canaries in coal mines, since they would be affected by toxic gases earlier than the miners, thus providing a biological warning system.”



lecture

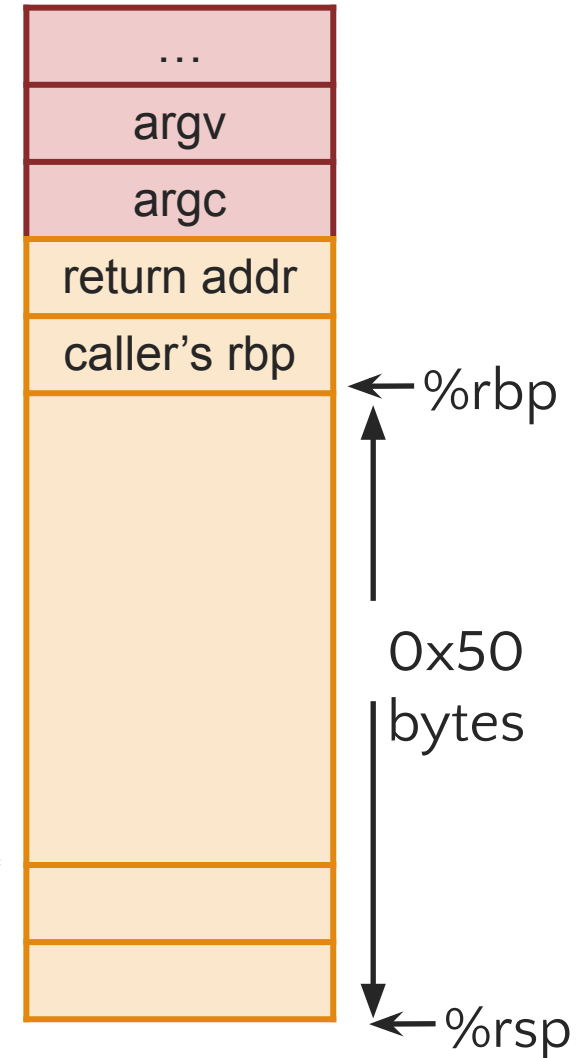
Reminder; Buffer Overflow

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    gets(buf);
}
```

Dump of assembly code for function main:

```
4004fd: push    %rbp
4004fe: mov     %rsp,%rbp
400501: sub     $0x50,%rsp
400505: mov     %rdi,-0x48(%rbp)
400508: mov     %rsi,-0x50(%rbp)
40050c: lea     -0x40(%rbp),%rax
400510: mov     %rax,%rdi
400518: callq   400400 <gets@plt>
```

Reg	Value
rax	buf
rdi	buf



Input

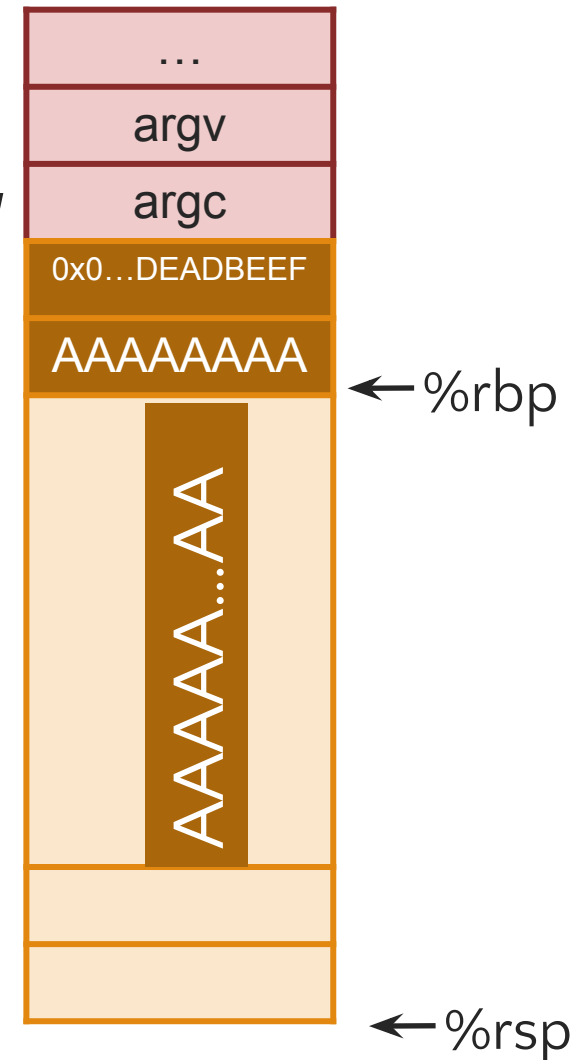
"A"x72 + "\xEF\xBE\xAD\xDE\x00\x00\x00\x00"

```
#include<string.h>
int main(int argc, char **argv) {
    char buf[64];
    gets(buf);
}

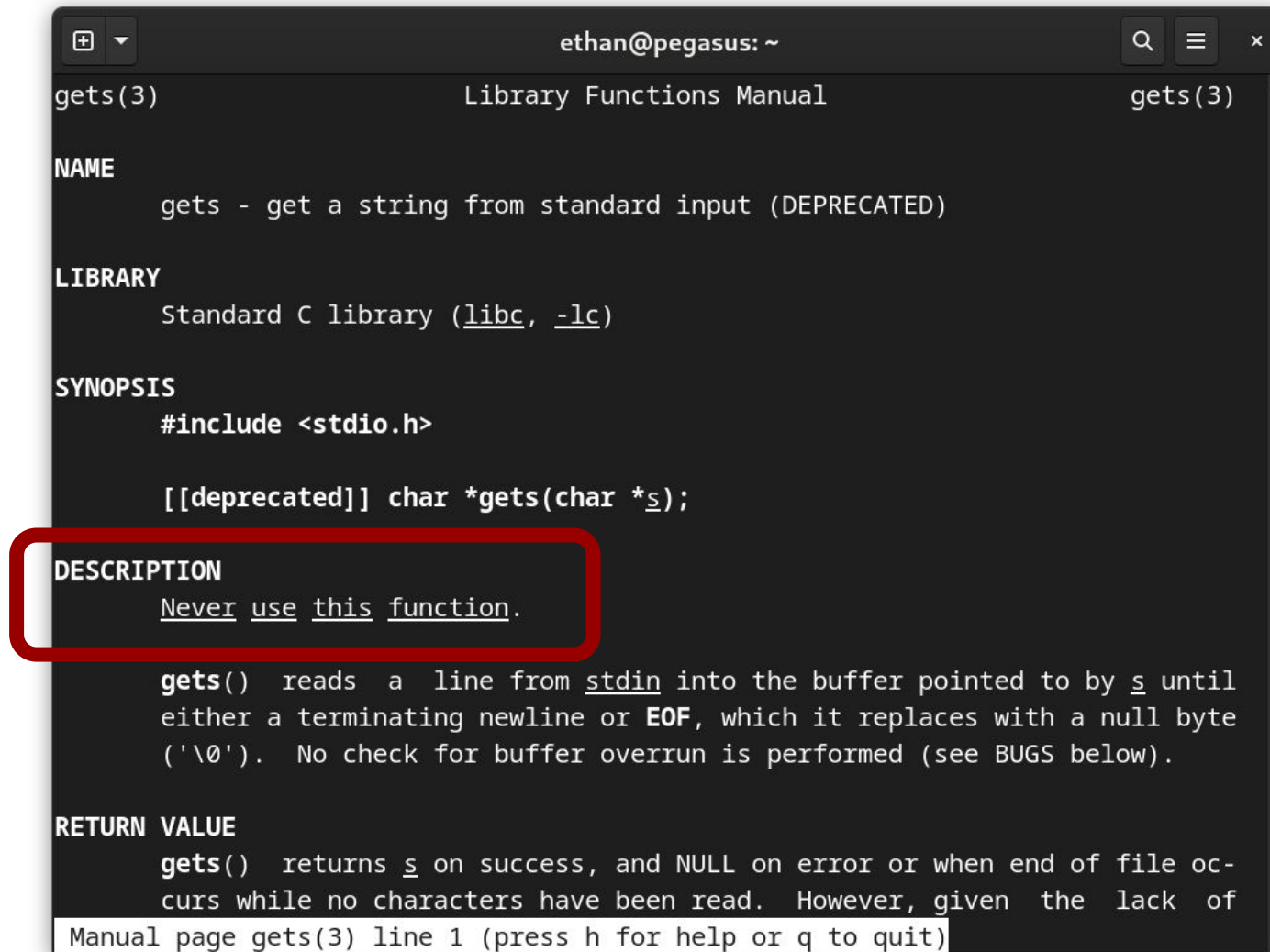
Dump of assembly code for function main:
4004fd: push    %rbp
4004fe: mov     %rsp,%rbp
400501: sub     $0x50,%rsp
400505: mov     %rdi,-0x48(%rbp)
400508: mov     %rsi,-0x50(%rbp)
40050c: lea     -0x40(%rbp),%rax
400510: mov     %rax,%rdi
400518: callq   400400 <gets@plt>
40051d: leaveq
40051e: retq
```

*corrupted
overwritten
overwritten*

Reg	Value
rax	buf
rdi	buf



PSA: Avoid "gets"



```
ethan@pegasus: ~
gets(3) Library Functions Manual gets(3)

NAME
    gets - get a string from standard input (DEPRECATED)

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <stdio.h>

    [[deprecated]] char *gets(char *s);

DESCRIPTION
    Never use this function.

    gets() reads a line from stdin into the buffer pointed to by s until
    either a terminating newline or EOF, which it replaces with a null byte
    ('\0'). No check for buffer overrun is performed (see BUGS below).

RETURN VALUE
    gets() returns s on success, and NULL on error or when end of file oc-
    curs while no characters have been read. However, given the lack of

Manual page gets(3) line 1 (press h for help or q to quit)
```

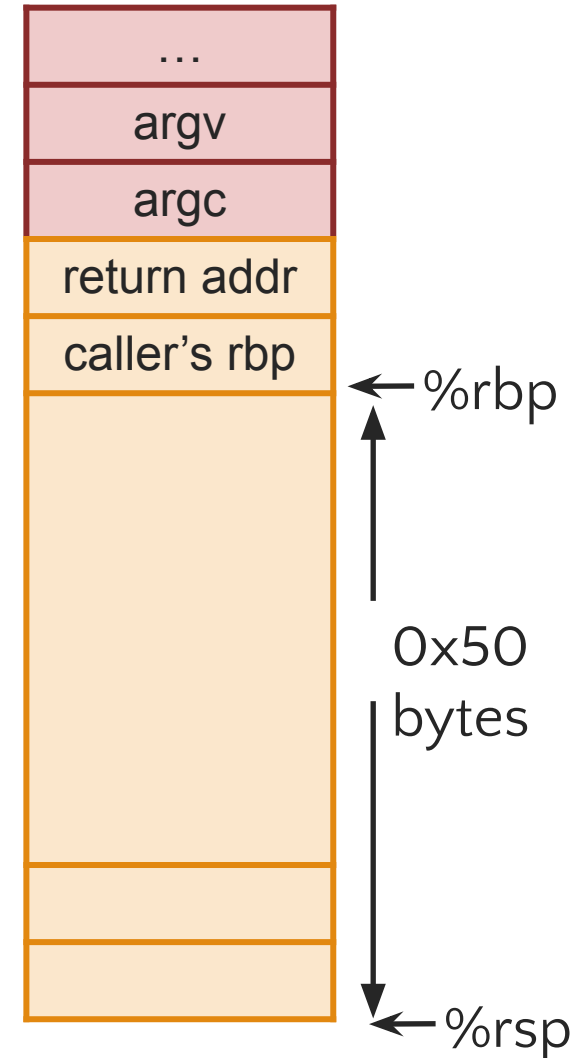

StackGuard

[Cowen et al. 1998]

Idea:

- prologue introduces a *canary word* between return addr and locals
- epilogue checks canary before function returns

Wrong canary => Overflow



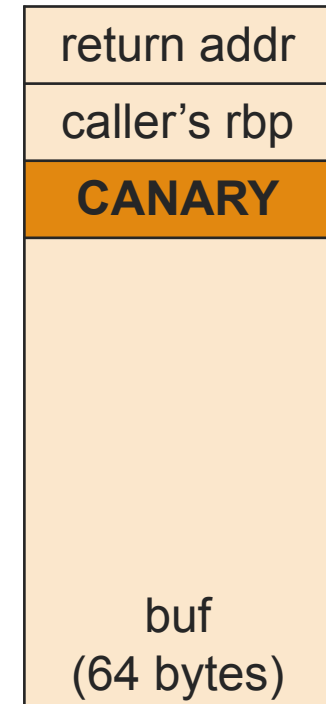
gcc Stack-Smashing Protector (ProPolice)

Dump of assembler code for function main:

```
4005a0: sub    $0x58,%rsp
4005a4: mov    %fs:0x28,%rax
4005ad: mov    %rax,0x48(%rsp)
4005b2: xor    %eax,%eax
4005b4: mov    %rsp,%rdi
4005b7: callq  4004a0 <gets@plt>
4005bc: mov    0x48(%rsp),%rdx
4005c1: xor    %fs:0x28,%rdx
4005ca: je     4005d1 <main+0x31>
4005cc: callq  400470 <__stack_chk_fail@plt>
4005d1: add    $0x58,%rsp
4005d5: retq
```

Compiled with `gcc -fstack-protector`

(you can also use `-fstack-protector-all` or `-fstack-protector-strong`)



Canary Should Be **HARD** to Forge



- Terminator canary
 - 4 bytes: 0,CR,LF,-1 (low->high)
 - terminate strcpy(), gets(), ...
- Random canary
 - 4 random bytes chosen at load time
 - stored in a guarded page
 - need good randomness

Proposed Defense Scorecard

Aspect	Defense
Performance	• Smaller impact is better
Deployment	• Can everyone easily use it?
Compatibility	• Doesn't break libraries
Safety Guarantee	• Completely secure vs. easy to bypass

Canary Scorecard

Aspect	Canary
Performance	• several instructions per function • time: a few percent on average • size: can optimize away in safe functions (but see MS08-067 *)
Deployment	• recompile suffices; no code change
Compatibility	• perfect—invisible to outside
Safety Guarantee	• <i>not really...</i>

[Shadow stack and canaries performance](#) (up to 50% slower - ouch!)

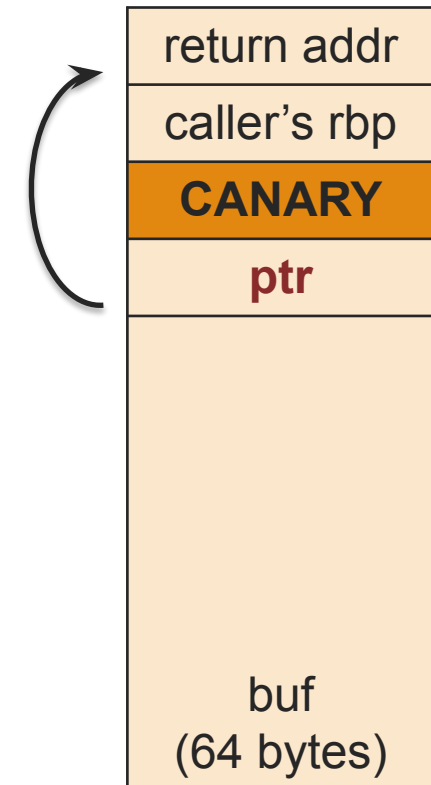


Defeating Canaries

Bypass: Data Pointer Subterfuge

Overwrite a data pointer *first*...

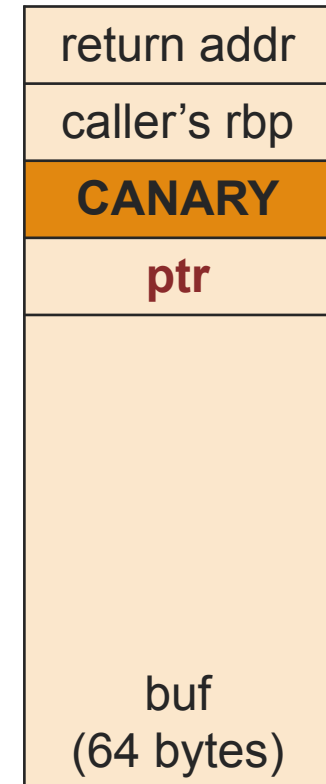
```
int *ptr;  
char buf[64];  
memcpy(buf, user1, large);  
*ptr = user2;
```



Bypass: Combine with a memory leak

Print out canary value first and use it in overwrite!

```
int *ptr;  
char buf[64];  
printf(user2);  
memcpy(buf, user1, large);
```



Canary Weakness

Check does ***not*** happen until epilogue...

- func ptr subterfuge
- C++ vtable hijack
- exception handler hijack
- ...

} PointGuard
} SafeSEH
SEHOP

} ProPolice
puts arrays
above others
when possible

└──────────┘
struct is fixed;
& what about heap?

Quiz

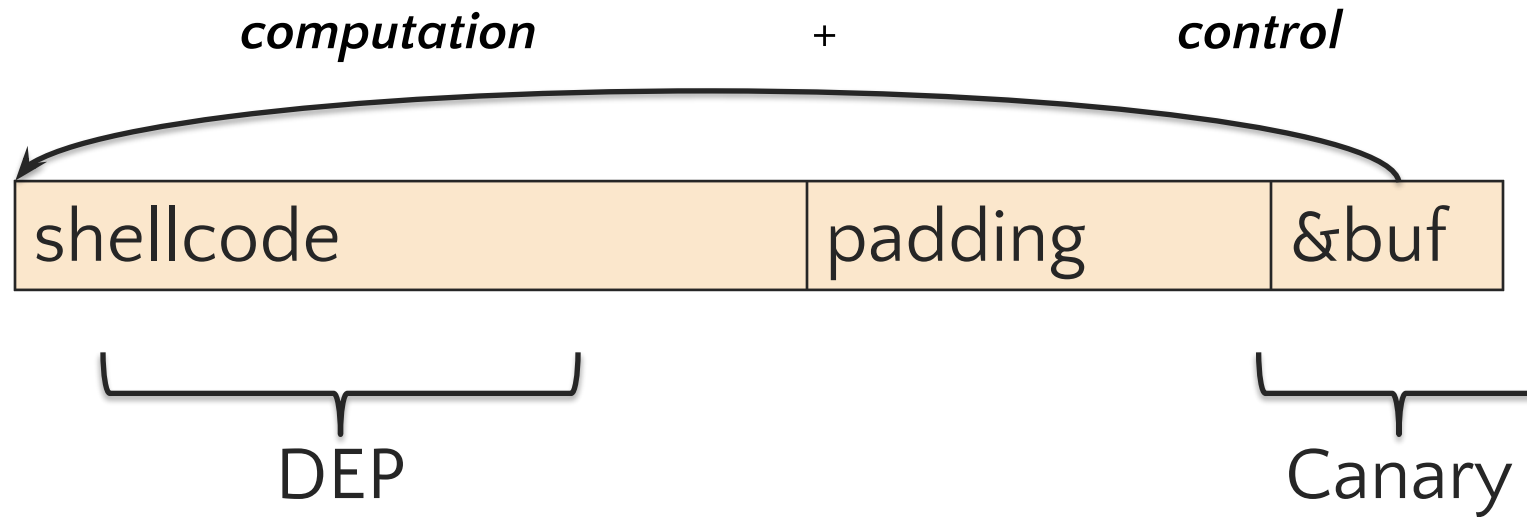
Which attack would be ***MOST*** effective at **hijacking control on a canary-protected machine?**

- A. Using a **single** `memcpy`-based buffer overflow of a local variable
- B. Using a format-string vulnerability **and** the “%n” specifier
- C. Using a format-string vulnerability **and** a targeted address specifier (e.g., “%9\$sBBBB\x47\xf7\xff\xff”)
- D. Using a format-string overflow of a local variable (e.g., “%80u\x3c\xd3\xff\xff”)

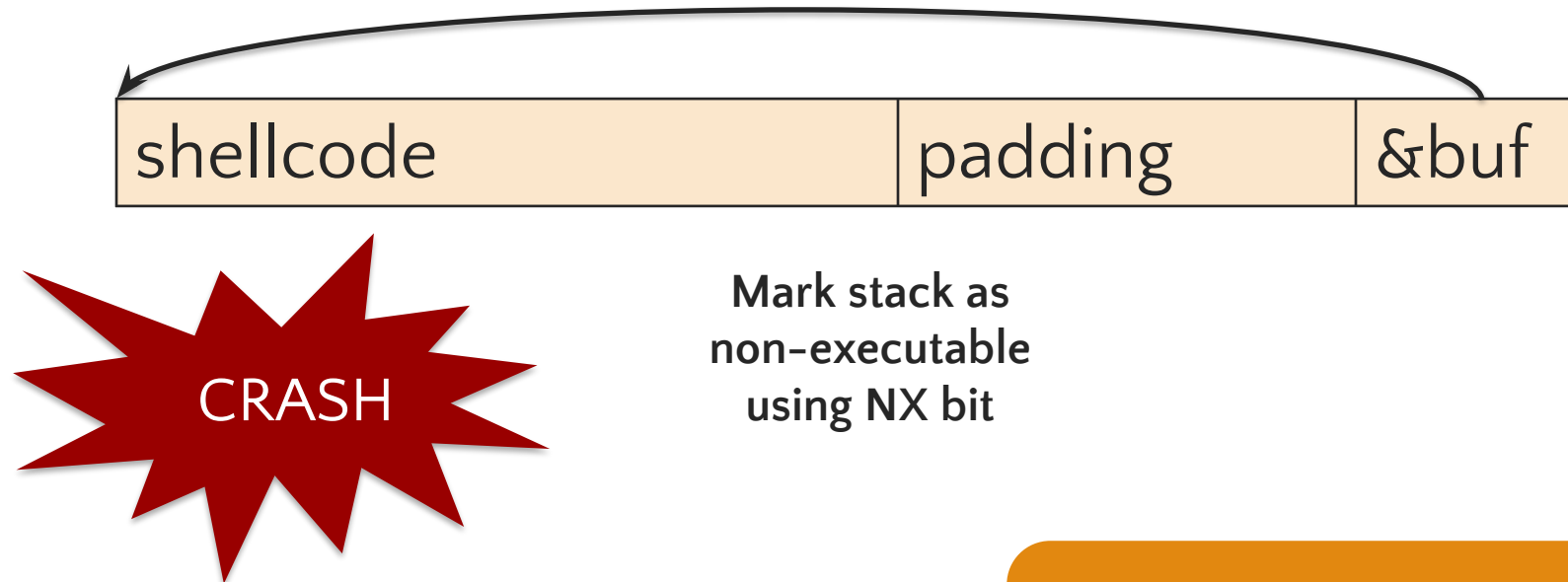
Introducing Data Execution Prevention (DEP) / No eXecute (NX)

Idea: maybe we shouldn't allow data to be executable

How to Defeat Exploits?



Data Execution Prevention



(still a Denial-of-Service attack!)

DEP prevents injected code on the stack from executing

DEP Scorecard

Aspect	Data Execution Prevention
Performance	• with hardware support: no impact • otherwise: reported to be <1% in PaX
Deployment	• kernel support (common on all platforms) • modules opt-in (less frequent in Windows)
Compatibility	• can break legitimate programs – Just-In-Time compilers – unpackers
Safety Guarantee	• code injected to NX pages never execute • <i>but code injection may not be necessary...</i>



Defeating DEP/NX

I need 2-3 volunteers

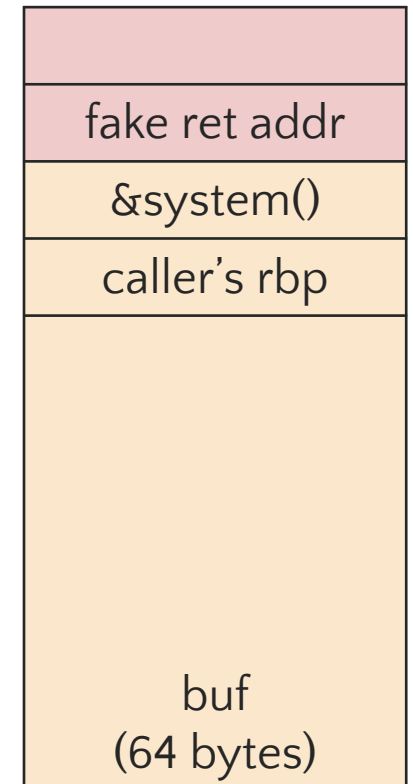
ssh ubuntu@44.201.87.16
ilovedi@uoa

Return-to-libc Attack

Overwrite return address with the address of a libc function

- setup fake return address
- put arguments (e.g. “/bin/sh”) in correct registers / memory
- ret will “call” libc function

No injected code!

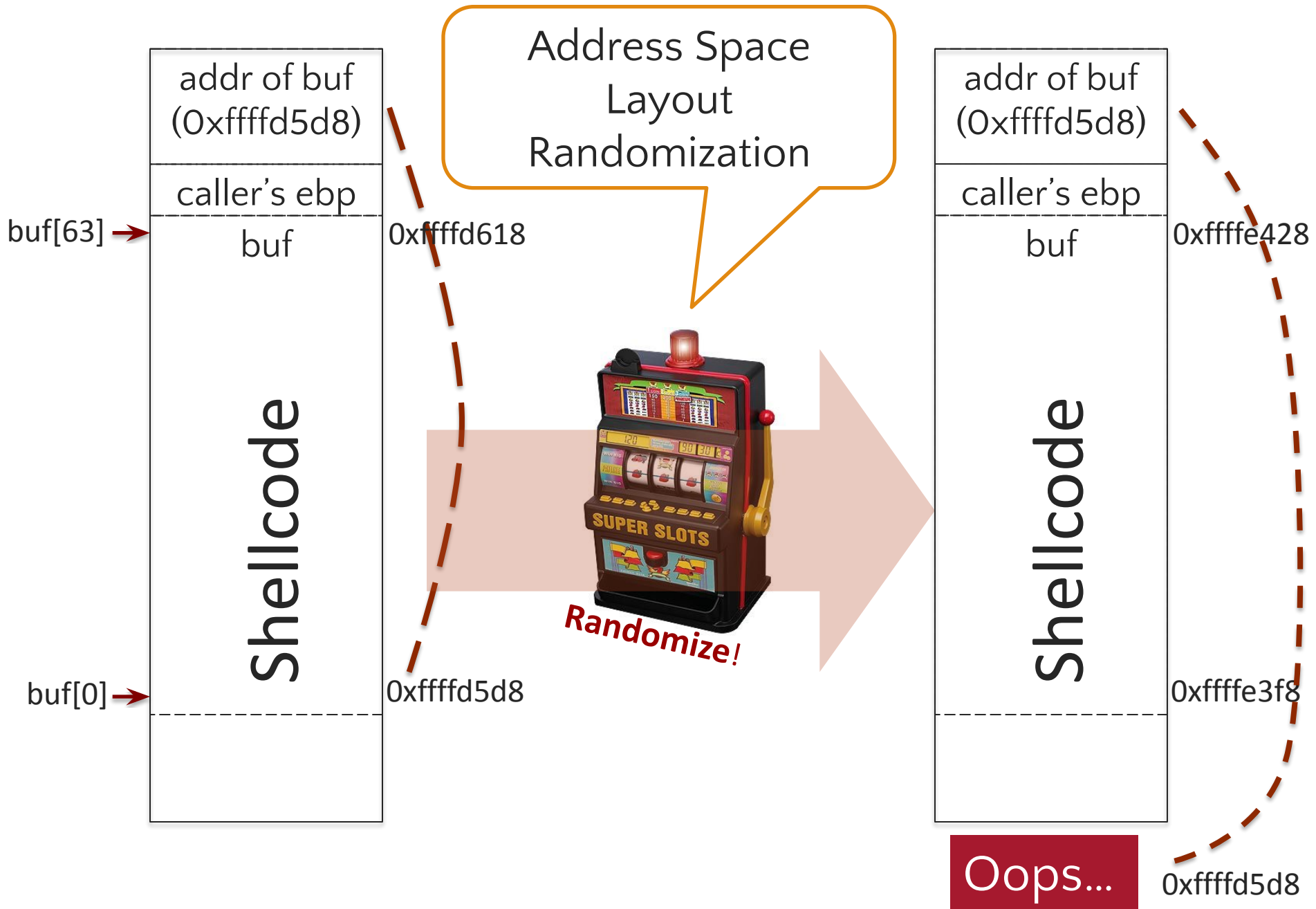


Introducing ASLR: Address Space Layout Randomization

Traditional exploits need precise addresses

- *stack-based overflows*: location of shell code
- *return-to-libc*: library addresses

- **Problem:** program's memory layout is fixed
 - stack, heap, libraries etc.
- **Solution:** randomize addresses of each region!





System Basics: The /proc Filesystem

The /proc filesystem (1/2)

proc is a virtual filesystem in Linux that provides a way to access system and process information.

- No actual files on disk everything generated dynamically in memory.

Key Features:

- Mounted at /proc
- Provides real-time system and process details
- Used for debugging, monitoring, and system configuration

Not sure how it works? Guess how we find out!

The /proc filesystem (2/2)

Typical useful examples:

Command	Description
/proc/cpuinfo	CPU details (cores, speed, vendor)
/proc/meminfo	Memory usage info
/proc/[PID]/cmdline	Command-line arguments of a process
/proc/[PID]/fd/	Open file descriptors of a process
/proc/[PID]/maps	Process memory layout
/proc/sys/	Kernel parameters (modifiable via sysctl)

Want to reference your own PID's resources? Use "self"! E.g., `cat /proc/self/cmdline`

An Important Concept: Core Files

A snapshot of a program's memory at the moment it crashes.

Typically includes:

- Stack, heap, globals, registers
- Memory mappings

Controlled via: `ulimit -c unlimited` and configured within:

`/proc/sys/kernel/core_pattern`

What does your `core_pattern` look like? Are cores sensitive?

Not sure about something? `man 5 core`!

Running cat Twice

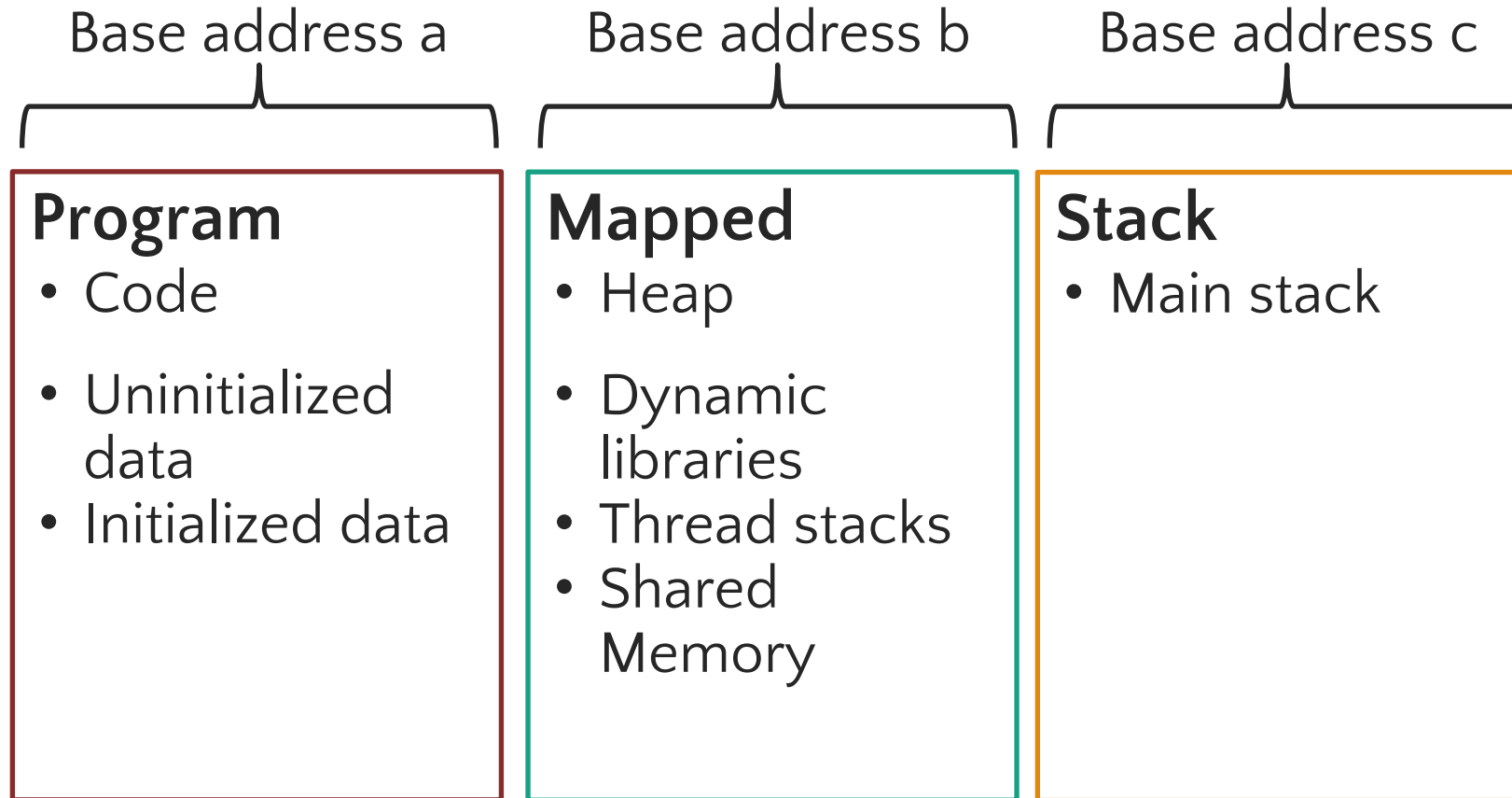
- Run 1

```
exploit:~# cat /proc/self/maps | egrep '(libc|heap|stack)'
082ac000-082cd000 rw-p 082ac000 00:00 0 [heap]
b7dfe000-b7f53000 r-xp 00000000 08:01 1750463 /lib/i686/cmov/libc-2.7.so
b7f53000-b7f54000 r--p 00155000 08:01 1750463 /lib/i686/cmov/libc-2.7.so
b7f54000-b7f56000 rw-p 00156000 08:01 1750463 /lib/i686/cmov/libc-2.7.so
bf966000-bf97b000 rw-p bffeb000 00:00 0 [stack]
```

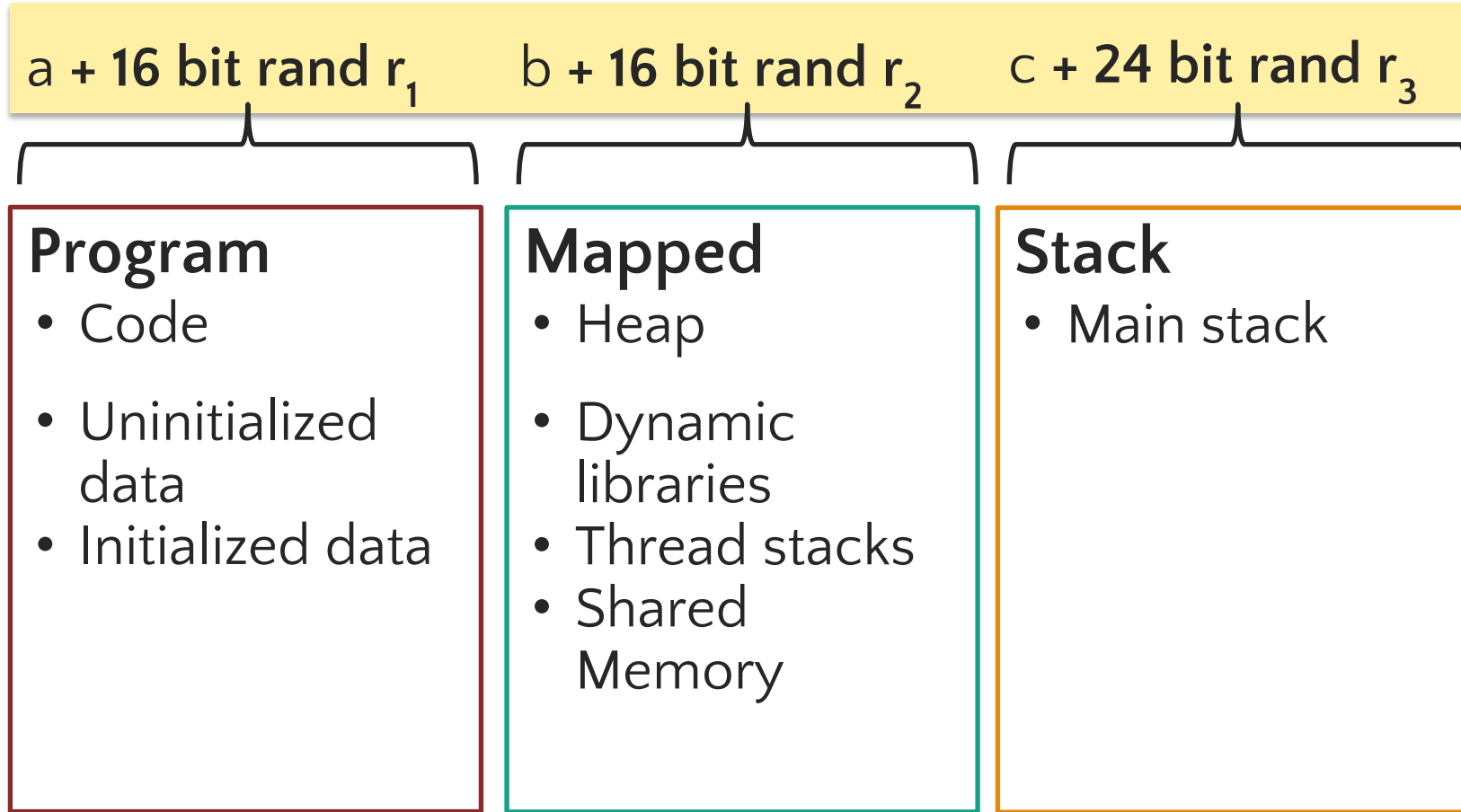
- Run 2

```
exploit:~# cat /proc/self/maps | egrep '(libc|heap|stack)'
086e8000-08709000 rw-p 086e8000 00:00 0 [heap]
b7d9a000-b7eef000 r-xp 00000000 08:01 1750463 /lib/i686/cmov/libc-2.7.so
b7eef000-b7ef0000 r--p 00155000 08:01 1750463 /lib/i686/cmov/libc-2.7.so
b7ef0000-b7ef2000 rw-p 00156000 08:01 1750463 /lib/i686/cmov/libc-2.7.so
bf902000-bf917000 rw-p bffeb000 00:00 0 [stack]
```


Memory



ASLR Randomization



* $\approx$ 16 bit random number of 32-bit system. More on 64-bit systems.

ASLR Scorecard

Aspect	Address Space Layout Randomization
Performance	• excellent—randomize once at load time
Deployment	• turn on kernel support (Windows: opt-in per module, but system override exists) • no recompilation necessary
Compatibility	• transparent to safe apps (position independent)
Safety Guarantee	• not good on x32, much better on x64 • <i>code injection may not be necessary...</i>

Checking which defenses are on

- Can be done by inspecting the binary
- Or using tools made for this – e.g., checksec (apt install)

```
$ checksec --file=/bin/ls
```

RELRO	STACK CANARY	NX	PIE	RPATH	RUNPATH	Symbols	FORTIFY	Fortified	Fortifiable	FILE
Full RELRO	Canary found	NX enabled	PIE enabled	No RPATH	No RUNPATH	No Symbols	Yes	6	18	/bin/ls

<http://slimm609.github.io/checksec.sh/>

Ευχαριστώ και καλή μέρα εύχομαι!

Keep hacking!